Step 1 — Two Standalone MCP-Connected Specialists — Solution

> Back to the step · Hint 1 · Hint 2

Story: two specialists, each opening its own MCP connection, each proven alone. No Supervisor, no graph yet. **If not:** a broken connection would surface inside Step 2's routing, and you'd be debugging two things at once.

Setup files

`requirements.txt`: `mcp`, `openai`, `langgraph`, `python-dotenv`, `pydantic`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

The two MCP servers

```
# servers/notes_server.py
# A minimal notes MCP server, seeded with a few Atlas notes.
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("notes-server")
NOTES = [
    "Atlas: beta ships next month, blocked on the auth rewrite.",
    "Atlas standup: QA found a login bug; fix in review.",
    "Atlas: design sign-off done; docs still pending.",
]

@mcp.tool()
def add_note(text):
    """Add a note to the notes store."""
    NOTES.append(text)
    return "Saved."

@mcp.tool()
def search_notes(query):
```

```
    """Search notes by keyword and return the matching lines."""
    hits = []
    for note in NOTES:
        if query.lower() in note.lower():
            hits.append(note)
    if not hits:
        return "No matching notes."
    return "\n".join(hits)

if __name__ == "__main__":
    print("notes server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

```
# servers/web_server.py
# A mocked web-fetch MCP server. A real deployment would point fetch_page at
# a real fetch/search MCP server (or the public reference fetch server); this
# is a hardcoded dict on purpose, for reliable, repeatable teaching.
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("web-server")
PAGES = {
    "https://intranet.example.com/atlas/status":
        "Atlas status page: 82% complete, launch on track for next month, "
        "two open risks (auth, docs).",
}

@mcp.tool()
def fetch_page(url):
    """Fetch a known internal status page by URL."""
    return PAGES.get(url, "No content available for this URL.")

if __name__ == "__main__":
    print("web server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

Settings, errors, logging

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
    connect_timeout: float

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        connect_timeout=float(os.getenv("MCP_CONNECT_TIMEOUT", "10")),
    )
```

```
# src/utils/logger.py
import logging
import os
import sys

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from openai import OpenAI

MODEL = "gpt-4o-mini"

def create_client(config):
    return OpenAI(api_key=config.openai_api_key)
```

The shared MCP specialist loop

Story: both specialists do the same thing — connect to *their* server, discover its tools, run a small ReAct loop. One shared `run_specialist` does it, returning the findings and an `unavailable` flag (the flag matters in Step 4). **If not:** you'd copy the whole connect-discover-loop into both agents.

```
# src/mcp_client.py
# The shared MCP specialist loop: connect to one server, discover its tools,
# run a ReAct loop, return (findings, unavailable). The connection is opened
# and closed inside this one async call (and this one task), which is why a
# specialist can run as a graph node without the teardown crossing tasks.
import asyncio
import json

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

from src.utils.logger import get_logger

MODEL = "gpt-4o-mini"

def _openai_tools(tools):
    schemas = []
    for tool in tools:
        schemas.append({
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or "",
                "parameters": tool.input_schema,
            },
        })
    return schemas

async def run_specialist(client, command, args, system, task,
                         connect_timeout=10.0):
```

```
    logger = get_logger(__name__)
    params = StdioServerParameters(command=command, args=args)
    try:
        async with stdio_client(params) as (read, write):
            async with ClientSession(read, write) as session:
                await asyncio.wait_for(
                    session.initialize(), timeout=connect_timeout
                )
                listed = await session.list_tools()
                tools = _openai_tools(listed.tools)
                messages = [
                    {"role": "system", "content": system},
                    {"role": "user", "content": task},
                ]
                for _ in range(5):
                    reply = client.chat.completions.create(
                        model=MODEL, messages=messages, tools=tools
                    ).choices[0].message
                    if not reply.tool_calls:
                        return reply.content or "", False
                    messages.append({
                        "role": "assistant", "content": reply.content,
                        "tool_calls": _calls_as_dicts(reply.tool_calls),
                    })
                    for call in reply.tool_calls:
                        arguments = json.loads(call.function.arguments)
                        result = await session.call_tool(
                            call.function.name, arguments
                        )
                        messages.append({
                            "role": "tool", "tool_call_id": call.id,
                            "content": result.content[0].text,
                        })
                return "(no answer)", False
    except Exception as e:
        logger.warning(f"specialist for {args} unavailable: {e}")
        return "", True

def _calls_as_dicts(tool_calls):
    out = []
    for call in tool_calls:
        out.append({
            "id": call.id, "type": "function",
            "function": {"name": call.function.name,
                         "arguments": call.function.arguments},
        })
    return out
```

The two specialists

```
# src/agents/notes_agent.py
from src.mcp_client import run_specialist

NOTES_SYSTEM = ("You are the Notes specialist. Use the notes tools to answer "
                "the task from the team's notes. Report only what you found.")

async def run_notes_agent(client, task, connect_timeout=10.0):
    findings, unavailable = await run_specialist(
        client, "python", ["servers/notes_server.py"],
        NOTES_SYSTEM, task, connect_timeout,
    )
    return findings, unavailable
```

```
# src/agents/web_agent.py
from src.mcp_client import run_specialist

WEB_SYSTEM = ("You are the Web specialist. Use the fetch tool to get the "
              "relevant page and summarize it. Report only what you fetched.")

async def run_web_agent(client, task, connect_timeout=10.0):
    findings, unavailable = await run_specialist(
        client, "python", ["servers/web_server.py"],
        WEB_SYSTEM, task, connect_timeout,
    )
    return findings, unavailable
```

main.py (standalone)

```
# main.py  (Step 1 version)
import asyncio
import sys

from src.agents.notes_agent import run_notes_agent
from src.agents.web_agent import run_web_agent
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import create_client

async def run_demo():
    try:
        config = load_config()
```

```
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
    client = create_client(config)
    notes, _ = await run_notes_agent(
        client, "What do my notes say about Project Atlas?"
    )
    print("Notes Agent:\n", notes)
    web, _ = await run_web_agent(
        client, "Fetch the Atlas status page "
        "(https://intranet.example.com/atlas/status) and summarize it.",
    )
    print("\nWeb Agent:\n", web)

def main():
    asyncio.run(run_demo())

if __name__ == "__main__":
    main()
```

The test

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "MCPCREW_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
```

```
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Expected output: `case 1..3 -> OK`. Running `main.py` prints the Notes Agent's Atlas findings and the Web Agent's fetched status summary — each specialist working on its own, before any Supervisor exists.